Assignment 4: Data Wrangling (Fall 2025)

Maeve Gualtieri-Reed

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Wrangling

Directions

1. Rename this file `<FirstLast>_A04_DataWrangling.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. Ensure that code in code chunks does not extend off the page in the PDF.

Set up your session

1a. Load the `tidyverse`, `lubridate`, and `here` packages into your session.

1b. Check your working directory.

1c. Read in all four raw data files associated with the EPA Air dataset, being sure to set string columns to be read in a factors. See the README file for the EPA air datasets for more information (especially if you have not worked with air quality data previously).

2. Add the appropriate code to reveal the dimensions of the four datasets.

```
#1a
library(tidyverse)
library(lubridate)
library(here)

#1b
here()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

```
#1c
EPA_O3_18 <- read.csv(
  file=here("Data/Raw/EPAair_O3_NC2018_raw.csv"),
  stringsAsFactors = TRUE
  )
```

```
EPA_O3_19 <- read.csv(
  file=here("Data/Raw/EPAair_O3_NC2019_raw.csv"),
  stringsAsFactors = TRUE
  )

EPA_PM25_18<- read.csv(
  file=here("Data/Raw/EPAair_PM25_NC2018_raw.csv"),
  stringsAsFactors = TRUE
)

EPA_PM25_19<- read.csv(
  file=here("Data/Raw/EPAair_PM25_NC2019_raw.csv"),
  stringsAsFactors = TRUE
)


#2
dim(EPA_O3_18)
```

```
## [1] 9737   20
```

```
dim(EPA_O3_19)
```

```
## [1] 10592    20
```

```
dim(EPA_PM25_18)
```

```
## [1] 8983   20
```

```
dim(EPA_PM25_19)
```

```
## [1] 8581   20
```

> All four datasets should have the same number of columns but unique record counts (rows). Do your datasets follow this pattern? Yes, all four datasets have 20 columns but different number of rows.

Wrangle individual datasets to create processed files.

3. Change the Date columns to be date objects.
4. Select the following columns: Date, DAILY_AQI_VALUE, Site.Name, AQS_PARAMETER_DESC, COUNTY, SITE_LATITUDE, SITE_LONGITUDE
5. For the PM2.5 datasets, fill all cells in AQS_PARAMETER_DESC with "PM2.5" (all cells in this column should be identical).
6. Save all four processed datasets in the Processed folder. Use the same file names as the raw files but replace "raw" with "processed".

```
#3
#Use lubridate mdy() to convert the input into a date class type
EPA_O3_18$Date<-mdy(EPA_O3_18$Date)
EPA_O3_19$Date<-mdy(EPA_O3_19$Date)
EPA_PM25_18$Date<-mdy(EPA_PM25_18$Date)
EPA_PM25_19$Date<-mdy(EPA_PM25_19$Date)
#confirm that the Date column is now a "Date"
class(EPA_O3_18$Date)
```

```
## [1] "Date"
```

```
class(EPA_O3_19$Date)
```

```
## [1] "Date"
```

```
class(EPA_PM25_18$Date)
```

```
## [1] "Date"
```

```
class(EPA_PM25_19$Date)
```

```
## [1] "Date"
```

```
#4
#Use the select() function in dplyr
EPA_O3_18<- select(EPA_O3_18, Date, DAILY_AQI_VALUE, Site.Name,
                   AQS_PARAMETER_DESC, COUNTY, SITE_LATITUDE, SITE_LONGITUDE)
EPA_O3_19<- select(EPA_O3_19, Date, DAILY_AQI_VALUE, Site.Name,
                   AQS_PARAMETER_DESC, COUNTY, SITE_LATITUDE, SITE_LONGITUDE)
EPA_PM25_18<- select(EPA_PM25_18, Date, DAILY_AQI_VALUE, Site.Name,
                   AQS_PARAMETER_DESC, COUNTY, SITE_LATITUDE, SITE_LONGITUDE)
EPA_PM25_19<- select(EPA_PM25_19, Date, DAILY_AQI_VALUE, Site.Name,
                   AQS_PARAMETER_DESC, COUNTY, SITE_LATITUDE, SITE_LONGITUDE)


#5
#use the mutate() function in dplyr
EPA_PM25_18<- mutate(EPA_PM25_18, AQS_PARAMETER_DESC="PM2.5")
EPA_PM25_19<- mutate(EPA_PM25_19, AQS_PARAMETER_DESC="PM2.5")

#6
write.csv(EPA_O3_18, row.names = FALSE,
          file = "./Data/Processed/EPAair_O3_NC2018_Processed.csv")
write.csv(EPA_O3_19, row.names = FALSE,
          file = "./Data/Processed/EPAair_O3_NC2019_Processed.csv")
write.csv(EPA_PM25_18, row.names = FALSE,
          file = "./Data/Processed/EPAair_PM25_NC2018_Processed.csv")
write.csv(EPA_PM25_19, row.names = FALSE,
          file = "./Data/Processed/EPAair_PM25_NC2019_Processed.csv")
```

Combine datasets

7. Combine the four datasets with `rbind`. Make sure your column names are identical prior to running this code.

8. Wrangle your new dataset with a pipe function (%>%) so that it fills the following conditions:

- Include only sites that the four data frames have in common:

"Linville Falls", "Durham Armory", "Leggett", "Hattie Avenue",
"Clemmons Middle", "Mendenhall School", "Frying Pan Mountain", "West Johnston Co.", "Garinger High School", "Castle Hayne", "Pitt Agri. Center", "Bryson City", "Millbrook School"

(the function `intersect` can figure out common factor levels - but it will include sites with missing site information, which you don't want...)

- Some sites have multiple measurements per day. Use the split-apply-combine strategy to generate daily means: group by date, site name, AQS parameter, and county. Take the mean of the AQI value, latitude, and longitude.
- Add columns for "Month" and "Year" by parsing your "Date" column (hint: `lubridate` package)
- Hint: the dimensions of this dataset should be 14,752 x 9.

9. Spread your datasets such that AQI values for ozone and PM2.5 are in separate columns. Each location on a specific date should now occupy only one row.

10. Call up the dimensions of your new tidy dataset.

11. Save your processed dataset with the following file name: "EPAair_O3_PM25_NC1819_Processed.csv"

```
#7
EPA_All <- rbind(EPA_O3_18, EPA_O3_19, EPA_PM25_18, EPA_PM25_19)

#8
EPA_All<- EPA_All %>%
  filter(Site.Name %in% c("Linville Falls", "Durham Armory", "Leggett", "Hattie Avenue",
  "Clemmons Middle", "Mendenhall School", "Frying Pan Mountain",
  "West Johnston Co.", "Garinger High School", "Castle Hayne",
  "Pitt Agri. Center", "Bryson City", "Millbrook School" )) %>%
  group_by(Date, Site.Name, AQS_PARAMETER_DESC, COUNTY) %>%  #only one row for
  #                                                          each iteration with
  #                                                          the same date/site/
  #                                                          county/AQS
  summarise(DAILY_AQI_VALUE = mean(DAILY_AQI_VALUE), #mean of AQI at each site
            SITE_LATITUDE = mean(SITE_LATITUDE),
            SITE_LONGITUDE = mean(SITE_LONGITUDE)) %>%
  mutate(Month = month(Date),
         Year = year(Date))
```

```
## `summarise()` has grouped output by 'Date', 'Site.Name', 'AQS_PARAMETER_DESC'.
## You can override using the `.groups` argument.
```

```
#9
#use pivot_wide to create a column for PM2.5 and a column for Ozone.
EPA_All<-pivot_wider(EPA_All,
                     names_from = AQS_PARAMETER_DESC,
                     values_from = DAILY_AQI_VALUE)

#10
dim(EPA_All)
```

```
## [1] 8976    9
```

```
#11
write.csv(EPA_All, row.names = FALSE,
          file = "./Data/Processed/EPAair_O3_PM25_NC1819_Processed.csv")
```

Generate summary tables

12. Use the split-apply-combine strategy to generate a summary data frame. Data should be grouped by site, month, and year. Generate the mean AQI values for ozone and PM2.5 for each group. Then, add a pipe to remove instances where mean **ozone** values are not available (use the function `drop_na` in your pipe). It's ok to have missing mean PM2.5 values in this result.

13. Call up the dimensions of the summary dataset.

```
#12
EPA_All_Summary <- EPA_All %>%
  group_by(Site.Name, Month, Year) %>% #group so that all rows with the same
  #                                    site, month, year become one row
  summarise(Mean_Ozone_AQI = mean(Ozone), #create a column for mean ozone
            Mean_PM2.5_AQI = mean(PM2.5)) %>% #create a column for mean PM2.5
  drop_na(Mean_Ozone_AQI) #any means that are NA are not included
```

```
## `summarise()` has grouped output by 'Site.Name', 'Month'. You can override
## using the `.groups` argument.
```

```
#13

dim(EPA_All_Summary)
```

```
## [1] 182   5
```

14. Why did we use the function `drop_na` rather than `na.omit`? Hint: replace `drop_na` with `na.omit` in part 12 and observe what happens with the dimensions of the summary date frame.

    Answer: The dataframe goes from 182 rows to 101. It looks like na.omit removes any rows with any NAs whereas drop_na only looks at the column we specify. Since we're ok with PM2.5 NA values, drop_NA is the right option in this case.